

JAVA PROGRAMMING

MODULE – 1

INTRODUCTION TO JAVA

Definition

Java is a class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It is intended to let application developers Write Once and Run Anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation. Java was developed by James Gosling at Sun Microsystems Inc. in May 1995 and later acquired by Oracle Corporation and is widely used for developing applications for desktop, web, and mobile devices.

Java is known for its simplicity, robustness, and security features, making it a popular choice for enterprise-level applications. Java applications are compiled to byte code that can run on any Java Virtual Machine. The syntax of Java is similar to C/C++.

Java makes writing, compiling, and debugging programming easy. It helps to create reusable code and modular programs.

Syntax

```
public class Classname
{
    public static void main(String[] args)
    {
        System.out.println("    ");
    }
}
```

Rules

- ☐ Save : Classname.java
- ☐ Compile : javac Classname.java
- ☐ Executions : java Classname

History of Java

Java was developed by **James Gosling** and his team at **Sun Microsystems** in **1991**. Initially, it was called **Oak**, named after a tree outside Gosling's office. However, due to trademark issues, it was later renamed **Java**, inspired by Java coffee beans.

The first official version, **Java 1.0**, was released in **1995** with the principle of "**Write Once, Run Anywhere**" (**WORA**), allowing Java programs to run on any system with a Java Virtual Machine (JVM). This made Java highly portable and popular among developers.

In **1999**, Java 2 (J2SE, J2EE, J2ME) was introduced, bringing major improvements in performance, security, and scalability. These versions helped Java expand into enterprise applications, mobile development, and embedded systems.

Sun Microsystems made Java **open-source** in **2006**, releasing it under the **GNU General Public License (GPL)**. This allowed developers worldwide to contribute to Java's development and led to its widespread adoption.

In **2010**, Oracle Corporation acquired **Sun Microsystems** and took over Java's development. Under Oracle, Java continued to evolve, and in **2017**, Oracle introduced a **six-month release cycle**, ensuring regular updates and new features.

Today, Java remains one of the most widely used programming languages, playing a crucial role in **web development, mobile apps (especially Android), enterprise software, and cloud computing**.

Features of java

1. Simple & Easy to Learn

Java is designed to be simple and beginner-friendly, especially for those familiar with C or C++. It removes complex features such as pointers, operator overloading, and multiple inheritance, which can cause confusion in other languages like C++. Java's syntax is straightforward and readable, making it easier to learn and write code efficiently. It follows a structured approach, reducing the chances of errors. Additionally, Java's extensive documentation and community support make it accessible for new learners.

2. Object-Oriented

Java follows the Object-Oriented Programming (OOP) paradigm, which promotes code reusability, scalability, and modularity. OOP concepts like classes, objects, inheritance, polymorphism, abstraction, and encapsulation help in organizing code efficiently. This approach makes it easier to manage large projects by breaking them into smaller, manageable units. OOP allows developers to create flexible and maintainable applications. It also enables easy collaboration among multiple developers by structuring programs into reusable components.

3. Robust

Java is a highly reliable and error-free programming language due to its strong memory management system. It uses automatic garbage collection to remove unused objects, reducing memory leaks. Java also implements exception handling, which prevents programs from crashing due to unexpected errors. Strong type checking ensures that variables and functions

are used correctly at both compile-time and runtime. These features make Java a stable and secure choice for developing large-scale applications.

4. Secure

Java provides high-level security features, making it a preferred choice for web and enterprise applications. It eliminates security risks by removing pointers, which can allow unauthorized access to memory in C and C++. The Java Security Manager restricts access to system resources, preventing malicious activities. Java programs run inside a controlled environment called the JVM sandbox, ensuring that they do not harm the system. These features make Java ideal for developing secure applications such as banking software and online transactions.

5. Platform-Independent

Java follows the "Write Once, Run Anywhere" (WORA) principle, meaning a program written in Java can run on any operating system without modification. Unlike C or C++, which require code to be recompiled for different platforms, Java uses the Java Virtual Machine (JVM) to execute bytecode. This makes Java highly portable and adaptable across Windows, macOS, Linux, and other OS. Developers can write cross-platform applications without worrying about compatibility issues. This feature is a key reason Java is widely used in enterprise and mobile applications.

6. High Performance

While Java is not as fast as C++, it achieves optimized execution through the Just-In-Time (JIT) compiler. The JIT compiler converts Java bytecode into native machine code at runtime, improving execution speed. Java also has an efficient garbage collection mechanism that manages memory automatically and reduces memory leaks. The language is designed with performance enhancements such as multithreading and caching techniques. With continuous improvements in JVM, Java has become significantly faster and more efficient for high-performance applications.

7. Distributed Computing

Java supports distributed computing, allowing applications to communicate over a network seamlessly. Technologies like Remote Method Invocation (RMI) and socket programming enable Java programs to interact with remote systems. This feature is widely used in cloud computing, web services, and enterprise applications that require data sharing. Java's ability to build robust, distributed applications makes it suitable for large-scale business and networking solutions. Many modern enterprise solutions use Java for client-server architectures and cloud-based platforms.

8. Multithreading

Java allows multiple tasks to execute simultaneously using multithreading, improving the efficiency of applications. Threads enable programs to run multiple operations in parallel without blocking each other. This is particularly useful in real-time applications such as gaming, multimedia, and web servers. Java's built-in thread management makes it easier to implement concurrency and optimize CPU utilization. With features like synchronization, Java ensures smooth execution of multiple processes without conflicts.

9. Dynamic & Extensible

Java is a highly dynamic language that allows developers to load new classes and modules at runtime. It supports external libraries and frameworks, making it easy to extend functionality as needed. Java's reflection API enables runtime modifications, improving flexibility and

adaptability. The language is continuously updated with new features and improvements. Developers can integrate Java applications with third-party libraries for enhanced capabilities, making it a versatile choice for software development.

10. Strong Community Support

Java has one of the largest and most active developer communities in the world. This ensures continuous updates, improvements, and availability of extensive learning resources. With strong community support, developers can find solutions to problems quickly through forums, tutorials, and open-source libraries. Java's popularity extends across multiple domains, including Android development, web applications, artificial intelligence, IoT, and big data. Companies and organizations worldwide rely on Java for building scalable and efficient applications. This widespread industry adoption ensures Java remains relevant and in high demand.

Applications of Java

1. Web Development

Java is widely used for developing dynamic and secure web applications. Frameworks like Spring, Hibernate, and Struts help build scalable enterprise applications. Java Servlets and JSP (Java Server Pages) enable server-side processing, making websites interactive. Many e-commerce, banking, and enterprise applications rely on Java for back-end development. Popular websites like LinkedIn and Amazon use Java for their web infrastructure.

2. Mobile App Development

Java is the primary language for Android app development. Android Studio, the official IDE for Android development, supports Java as its main programming language. Mobile applications in gaming, social media, and utilities are built using Java. The cross-platform nature of Java allows developers to create Android apps that run on different devices. Many popular Android apps, including those on the Google Play Store, are developed using Java.

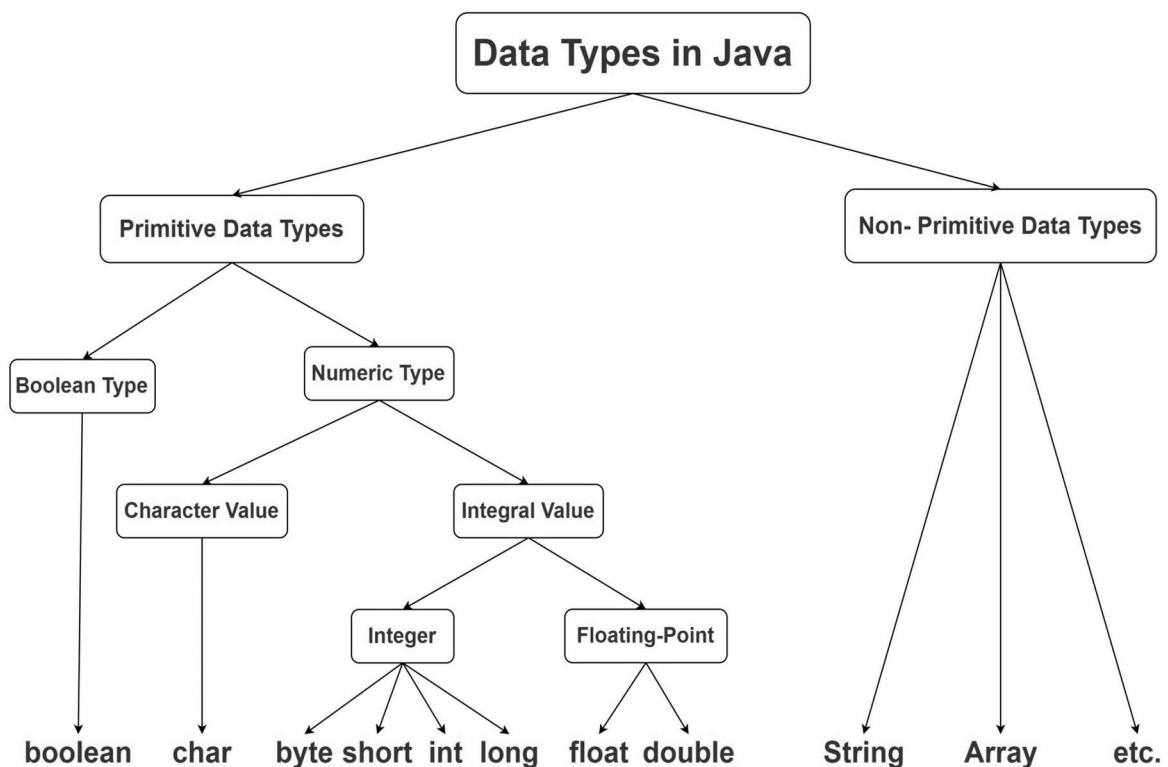
3. Enterprise Applications

Java is extensively used in large-scale enterprise software due to its reliability and security. It powers Customer Relationship Management (CRM), Enterprise Resource Planning (ERP), and banking systems. Java Enterprise Edition (Java EE) provides a robust platform for enterprise-level applications. Many multinational corporations, including banks and insurance companies, use Java-based software. The ability to handle large databases and transactions makes Java a preferred choice for businesses.

4. Cloud Computing

Java plays a crucial role in cloud-based applications and services. Many cloud providers, including AWS, Google Cloud, and Microsoft Azure, support Java for developing cloud solutions. Java's distributed computing capabilities make it ideal for cloud applications that require scalability. Companies use Java to build Software-as-a-Service (SaaS) and Platform-as-a-Service (PaaS) applications. Java frameworks like Spring Boot are commonly used for deploying microservices in the cloud.

Data Types in Java:-



Data types refer to the different sizes and values that can be stored in the variable.

Two types of data type are in Java programming:

A.Primitive data types : The primitive data types consist of int, float, boolean, byte, short, long, char and double.

B.Non-primitive data types: The non-primitive data types include arrays, interfaces and class etc.

Java Primitive Data Types:-

There are following primitive data types available in Java programming language.

- (1) Byte data type
- (2) Boolean data type
- (3) Int data type
- (4) Short data type
- (5) Char data type

(6) Double data type

(7) Float data type

(8) Long data type

(1) Byte Data Type: It is the first data type with least memory size allocation which can be used for numbers of small ranges.

(a) The memory size is allocated 1 byte.

(b) It can represent a total 256(2).

(c) byte can represent from 0 to 127 on positive side (as zero is positive number per programming) and on the negative side it can represent the number -1 to 128.

(d) The default value for byte is zero (0).

Example:- byte a1 = 10;

(2) Boolean Data Type: - The boolean data type is a one bit information. Only two possible values are of Boolean data type. Which are true and false.

(a) It has not something range of values of the variable.

(b) The values true or false are case-sensitive keywords.

Example:- boolean a = false; boolean b=true;

(3) Int Data Type:-

The int data type is a 32-bits signed type. Minimum value of int data type is - 2,147,483,648 and maximum value of int data type is 2,147,483,647 precision type.

(a) Its default value is 0.

(b) On the positive side 0 to 2,147,483,647 and on the negative side -1 to 2,147,483,647

(c) It can represent a total of 4,294,967,296

Example:- int a = 100000;

int b = -200000;

(4) Short Data Type:-

The short data type is a 16-bit signed type. Its value-range lies between - 32,768 to 32,767. Minimum value of short is -32,768 and maximum value of short is 32,767.

(a) Its default value is 0.

(b) It can represent total 65536(2¹⁶) numbers.

Example:- short s = 10000;

(5) Char Data Type:-

It has a single 16-bit Unicode character. Value-range of char data type lies between -127 to 127. The char data type is used to store characters.

(a) It stores a single character such as a letter, number and punctuation mark or other symbol.

(b) Characters are a single letter enclosed in single quotes.

Example:- char b = 'A'; char a = '#';

(6) Double Data Type:-

double data type is a 64 bits signed type. Its value range is unlimited. The double data type is generally used for decimal (points) values just like float. The double data type does not use for precise values, such as currency.

(a) Its default value is 0.0d.

Example:- double d1 = 122.39;

(7) Float Data Type:-

The float data type has a single-precision 32-bits types and its value range is unlimited.

(a) Its default value is 0.0F.

Example:- float f1 = 134.5f;

(8) Long Data Type:-

It has a 64-bit two's complement integer. Minimum value long data type is -9,223,372,036,854,775,808 and maximum value of long data type is 9,223,372,036,854,775,807.

(a) Its default value is 0.

Example:- long a = 100000L;

Java Non-Primitive Data Types:-

There are following non- primitive data types available in Java programming language.

(1) **Array**:- An array is the collection of homogeneous (or similar types) data type.

(a) An array is an object that holds a fixed number of values of homogeneous or similar data-type.

(b) The length of an array is assigned when the array is created and after creation, its length is fixed.

Example:- `int a[]=new int[6];`

(2) **Class:** - A class is a “user defined data type” from which objects are created of class. In general, class declarations can include components. And it consists of data and methods in the form of a unit.

(a) **Modifiers:** - A class can be public or default access.

(b) **Class name:** - The name of class should begin with an initial capital letter.

(c) **Body:** - The class body is enclosed by braces {}.

Example: - `public class car`

`{`

`Public:`

`char color;`

`double model;`

`Public void gear (); // behavior of a car`

`}`

(3) **Interface:** - An interface is basically a kind of class. So an interface is a collection of “methods” without actual definitions and “variables”.

Thus it is the responsibility of the class that to define and implement the codes of these methods.

Example:-

`interface item`

`{`

`Static final int code=101;`

`Static final string name =”fan”;`

`Void display ();`

`}`

(4) String

A String is a sequence of characters stored as an object.

It is not a primitive data type because it is a class with built-in methods.

Strings are immutable, meaning their values cannot be changed once created.

We can create a string using double quotes ("Hello") or the new keyword.

Example:

```
String greeting = "Hello, World!";  
  
System.out.println(greeting); // Output: Hello, World!
```

(5) Enum

An enum is used to define a set of fixed constant values.

It makes the code more readable and prevents invalid values.

Enums are useful for representing things like days of the week, colors, or directions.

Each value in an enum is a constant.

Example:

```
enum Day { SUNDAY, MONDAY, TUESDAY }  
  
public class Main  
{  
    public static void main(String[] args)  
    {  
        Day today = Day.MONDAY;  
        System.out.println("Today is " + today); // Output: Today is MONDAY  
    }  
}
```

Variables in Java

In Java, variables are used to store data in memory. Each variable has a data type, a name, and a value. Variables act as containers that hold information that can be referenced and manipulated within a program.

Types of Variables in Java

Java has three main types of variables:

1. Local Variables

Declared inside a method, constructor, or block.

Scope is limited to the block where it is declared.

Must be initialized before use, as Java does not assign default values to local variables.

Example:

```
public class Example
{
    public void show()
    {
        int num = 10; // Local variable
        System.out.println(num);
    }
}
```

2. Instance Variables (Non-Static Variables)

Declared inside a class but outside any method.

Each object of the class has its own copy of instance variables.

Have default values (0 for numbers, null for objects, false for booleans).

Example:

```
class Student
{
    String name; // Instance variable
```

```
int age;

void display() {

    System.out.println("Name: " + name + ", Age: " + age);

}

}
```

3. Static Variables (Class Variables)

Declared using the static keyword.

Shared by all objects of the class (a single copy is created for all instances).

Stored in static memory.

Example:

```
class Employee

{

    static String company = "TechCorp"; // Static variable

    void showCompany()

    {

        System.out.println("Company: " + company);

    }

}
```

Variable Naming Rules in Java

- Must start with a letter (a-z, A-Z), underscore _, or dollar \$.
- Cannot use reserved keywords as variable names.
- Case-sensitive (e.g., age and Age are different variables).
- CamelCase convention is recommended (e.g., studentName, totalAmount).

Operators in Java

Operators in Java are used to perform operations on variables and values. They help in mathematical calculations, logical decisions, and bitwise operations. Below are the main types of operators in Java with brief explanations.

1. Arithmetic Operators

Arithmetic operators perform basic mathematical operations like addition, subtraction, multiplication, division, and modulus. They help in performing calculations efficiently. The modulus operator (%) gives the remainder of a division. These operators work on numeric data types like int, float, and double.

Example:

```
int a = 10, b = 5;  
System.out.println(a + b); // Output: 15
```

2. Relational (Comparison) Operators

These operators compare two values and return true or false based on the comparison. They are commonly used in decision-making and loops. Operators like == check equality, while != checks inequality. Greater than (>) and less than (<) help in ordering values.

Example:

```
int a = 10, b = 5;  
System.out.println(a > b); // Output: true
```

3. Logical Operators

Logical operators combine multiple conditions and return a boolean value (true or false). The AND (&&) operator returns true if both conditions are true. The OR (||) operator returns true if at least one condition is true. The NOT (!) operator inverts the boolean value.

Example:

```
boolean x = true, y = false;  
System.out.println(x && y); // Output: false
```

4. Bitwise Operators

Bitwise operators perform operations at the binary level. The AND (&) and OR (|) operators compare bits and return a new value. The XOR (^) operator returns 1 if the bits are different. The NOT (~) operator inverts all bits.

Example:

```
int a = 5, b = 3;  
System.out.println(a & b); // Output: 1
```

5. Assignment Operators

Assignment operators assign values to variables. The = operator is used for direct assignment. Compound assignment operators like +=, -=, and *= modify the value while assigning. These operators help in reducing code length. They are commonly used in loops and mathematical expressions.

Example:

```
int a = 10;  
a += 5; // Same as a = a + 5;  
System.out.println(a); // Output: 15
```

6. Unary Operators

Unary operators work on a single operand. The ++ (increment) and -- (decrement) operators increase or decrease the value by one. The unary minus (-) changes the sign of a number. The NOT (!) operator flips a boolean value (true to false and vice versa).

Example:

```
int a = 5;
System.out.println(++a); // Output: 6
```

7. Ternary Operator

The ternary operator is a shorthand for an if-else statement. It evaluates a condition and returns one of two values. It follows the syntax `condition ? value_if_true : value_if_false`. This operator is useful for concise decision-making in assignments.

Example:

```
int a = 10, b = 5;
int min = (a < b) ? a : b;
System.out.println(min); // Output: 5
```

8. Shift Operators

Shift operators move the bits of a number left (<<) or right (>>). Left shift (<<) multiplies the number by 2^n . Right shift (>>) divides the number by 2^n . The unsigned right shift (>>>) fills the left bits with zeros.

Example:

```
int a = 8;
System.out.println(a << 2); // Output: 32 (8 * 2^2)
```

Control Structures in Java

Control structures in Java manage the flow of execution in a program. They allow decisions to be made, loops to be executed, and specific parts of code to be skipped or jumped to.

1. Selection (Decision-Making) Statements

Selection statements allow a program to make decisions and execute different code blocks based on conditions.

a) if Statement

Executes a block of code if the condition is true.

Syntax:

```
if (condition)
{
    // Code executes if condition is true
}
```

Example:

```
int num = 10;
if (num > 0)
{
    System.out.println("Positive number");
}
```

// Output: Positive number

b) if-else Statement

Executes one block if the condition is true, otherwise executes another block.

Syntax:

```
if (condition)
{
    // Code executes if condition is true
}
else
{
    // Code executes if condition is false
}
```

Example:

```
int num = -5;
if (num > 0)
{
    System.out.println("Positive number");
}
else
{
    System.out.println("Negative number");
}
// Output: Negative number
```

c) if-else if Ladder

Used when multiple conditions need to be checked sequentially.

Syntax:

```
if (condition1)
{
    // Executes if condition1 is true
}
else if (condition2)
{
    // Executes if condition2 is true
}
Else
{
    // Executes if none of the conditions are true
}
```

Example:

```
int marks = 85;
if (marks >= 90)
{
```

```

    System.out.println("Grade: A");
}
else if (marks >= 75)
{
    System.out.println("Grade: B");
}
else
{
    System.out.println("Grade: C");
}
// Output: Grade: B

```

d) Nested if Statement

An if statement inside another if statement.

Syntax:

```

if (condition1)
{
    if (condition2)
    {
        // Code executes if both conditions are true
    }
}

```

Example:

```

int num = 15;
if (num > 0) {
    if (num % 2 == 0)
    {
        System.out.println("Positive Even Number");
    }
Else
{
    System.out.println("Positive Odd Number");
}
}
// Output: Positive Odd Number

```

e) switch Statement

Used when multiple conditions are checked against a single variable.

Syntax:

```

switch (variable)
{
    case value1:
        // Code for value1
        break;
    case value2:
        // Code for value2
        break;
}

```

```
    default:
        // Code if none of the cases match
}
```

Example:

```
int day = 2;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    default:
        System.out.println("Other day");
}
// Output: Tuesday
```

2.Iteration (Looping) Statements

Loops are used to execute a block of code multiple times.

a) for Loop

Used when the number of iterations is known.

Syntax:

```
for (initialization; condition; update)
{
    // Code to be executed
}
```

Example:

```
for (int i = 1; i <= 5; i++)
{
    System.out.println("Hello " + i);
}
// Output: Hello 1, Hello 2, ..., Hello 5
```

b) while Loop

Executes as long as the condition is true.

Syntax:

```
while (condition)
{
    // Code to be executed
}
```

Example:

```
int i = 1;
while (i <= 5) {
    System.out.println("Count: " + i);
}
```



```
i++;  
}  
// Output: Count: 1, Count: 2, ..., Count: 5
```

c) do-while Loop

Executes the code at least once before checking the condition.

Syntax:

```
do {  
    // Code to be executed  
}  
while (condition);
```

Example:

```
int i = 1;  
do {  
    System.out.println("Number: " + i);  
    i++;  
} while (i <= 3);  
// Output: Number: 1, Number: 2, Number: 3
```

d) Enhanced for Loop (for-each Loop)

Used for iterating over arrays and collections.

Syntax:

```
for (dataType variable : array)  
{  
    // Code to be executed  
}
```

Example:

```
int[] numbers = {1, 2, 3, 4, 5};  
for (int num : numbers)  
{  
    System.out.println(num);  
}  
// Output: 1, 2, 3, 4, 5
```

3. Jump Statements

Jump statements control the flow by breaking or continuing loops.

a) break Statement

Exits the loop immediately.

Syntax:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {
```

```
        break; // Stops loop when i is 3
    }
    System.out.println(i);
}
```

Example:

```
for (int i = 1; i <= 5; i++)
{
    if (i == 3) {
        break;
    }
    System.out.println(i);
}
// Output: 1, 2
```

b) continue Statement

Skips the current iteration and moves to the next one.

Syntax:

```
for (int i = 1; i <= 5; i++)
{
    if (i == 3)
    {
        continue; // Skips iteration when i is 3
    }
    System.out.println(i);
}
```

Example:

```
for (int i = 1; i <= 5; i++)
{
    if (i == 3)
    {
        continue;
    }
    System.out.println(i);
}
// Output: 1, 2, 4, 5
```

c) return Statement

Exits from a method and returns a value.

Syntax:

return value;

Example:

```
public class Example
{
```

```

public static int sum(int a, int b)
{
    return a + b; // Returns sum
}

public static void main(String[] args)
{
    int result = sum(5, 3);
    System.out.println("Sum: " + result);
}
// Output: Sum: 8

```

These control structures are essential in Java for making decisions, repeating actions, and managing program flow effectively.

Object Oriented Programming Concepts

Object-Oriented Programming (OOP) is a programming paradigm that uses objects and classes to structure a program. It focuses on data (objects) rather than logic and provides a modular and reusable approach to software development.

Key Principles of OOP

1. **Encapsulation** – Wrapping data and methods into a single unit (class) and restricting direct access.
2. **Abstraction** – Hiding implementation details and exposing only essential features.
Inheritance – Acquiring properties and behaviour from an existing class.
3. **Polymorphism** – The ability to take multiple forms (method overloading and overriding).

1. Class

A class is a blueprint or template for creating objects. It defines attributes (variables) and behaviors (methods).

A class does not occupy memory until an object is created.

Types of Classes in Java:

Predefined classes – Provided by Java (e.g., Scanner, String, System)

User-defined classes – Created by the user (e.g., Student, Car, Employee)

Syntax

```

class ClassName
{
    // Data members (variables)
    // Methods (functions)
}

```

2. Object

An object is an instance of a class. It is a real-world entity with attributes and behaviors.

Objects are stored in heap memory.

They are created using the new keyword

Syntax

```
ClassName obj = new ClassName();
```

Example:

```
class Car
{
    String model = "Toyota";
}
public class Main
{
    public static void main(String[] args)
    {
        Car myCar = new Car();
        System.out.println(myCar.model); // Output: Toyota
    }
}
```

3. Encapsulation

Encapsulation is the binding of data and methods into a single unit while restricting direct access.

Private variables prevent external modification.

Use getters and setters to access and modify private data.

Example

```
class Student
{
    private int age; // Private variable
    public void setAge(int a)
    { // Setter
        if (a > 0)
        {
            age = a;
        }
    }

    public int getAge() { // Getter
        return age;
    }
}
public class Main
{
    public static void main(String[] args)
    {
        Student s = new Student();
        s.setAge(18);
        System.out.println("Student age: " + s.getAge()); // Output: 18
    }
}
```

4. Abstraction

Abstraction is the process of hiding implementation details and showing only necessary features.

Achieved using:

Abstract Class (using abstract keyword)

Interface (using interface keyword)

Abstract Class Example

```
abstract class Animal
{
    abstract void makeSound(); // Abstract method (no implementation)
}

class Dog extends Animal
{
    void makeSound()
    {
        System.out.println("Dog barks");
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Animal myDog = new Dog();
        myDog.makeSound(); // Output: Dog barks
    }
}
```

Interface Example

```
interface Animal
{
    void makeSound(); // Method without body
}

class Cat implements Animal
{
    public void makeSound()
    {
        System.out.println("Cat meows");
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Animal myCat = new Cat();
        myCat.makeSound(); // Output: Cat meows
    }
}
```

5. Inheritance

Inheritance allows a child class to acquire properties and behaviors of a parent class, promoting code reusability.

```
// Parent class
class Animal
{
    void eat()
    {
        System.out.println("I can eat");
    }
}

// Child class
class Dog extends Animal {
    void bark() {
        System.out.println("I can bark");
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Dog myDog = new Dog();
        myDog.eat(); // Inherited method from Animal
        myDog.bark(); // Dog's own method
    }
}
```

6. Polymorphism

Polymorphism means one function, multiple behaviors.
It is achieved through:

```
class MathOperations
{
    int add(int a, int b)
    {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main
{
    public static void main(String[] args)
    {
        MathOperations math = new MathOperations();
        System.out.println(math.add(5, 10)); // Output: 15
        System.out.println(math.add(5.5, 2.5)); // Output: 8.0
    }
}
```

Method Overloading

Method overloading is a feature in Java that allows multiple methods in the same class to have the same name but with different parameters. It improves code readability and reusability.

Method overloading occurs when:

Methods have the same name but different parameter lists (different number, type, or order of parameters).

It allows multiple methods to perform similar functions but with different input values.

Rules for Method Overloading

- Different Number of Parameters – The number of arguments must be different.
- Different Types of Parameters – The data types of arguments must be different.
- Different Order of Parameters – The sequence of parameters can be changed.
- Return Type Does Not Matter – Overloading cannot be achieved by changing only the return type.

Example :

```
class OverloadExample {  
    // Method with one parameter  
    void display(int a) {  
        System.out.println("Integer: " + a);  
    }  
  
    // Method with two parameters  
    void display(int a, double b) {  
        System.out.println("Integer: " + a + " and Double: " + b);  
    }  
  
    // Method with different parameter type  
    void display(String msg) {  
        System.out.println("Message: " + msg);  
    }  
  
    public static void main(String args[])  
    {  
        OverloadExample obj = new OverloadExample();  
        obj.display(5);  
        obj.display(10, 20.5);  
        obj.display("Hello Java");  
    }  
}
```

OUTPUT

Integer: 5

Integer: 10 and Double: 20.5

Message: Hello Java

Math class in java

- The Math class in Java is a part of the java.lang package and provides a collection of static methods for performing mathematical operations. It is a final class, meaning it cannot be instantiated or extended.
- The Math class includes methods for basic arithmetic, trigonometry, logarithmic calculations, exponentiation, and rounding. It also provides mathematical constants like PI ($\pi = 3.14159\dots$) and E (Euler's number, approximately 2.718).
- Since all the methods in the Math class are static, they can be accessed directly using Math.methodName() without creating an object. Commonly used methods include Math.abs() for absolute values, Math.sqrt() for square roots, Math.pow() for exponentiation, and Math.random() for generating random numbers.
- The class also provides trigonometric functions such as Math.sin(), Math.cos(), and Math.tan(), which work with angles in radians.
- The Math class is widely used in scientific computations, game development, financial applications, and any scenario where mathematical calculations are required.

```
class MathExample
{
    public static void main(String args[])
    {
        System.out.println("Absolute: " + Math.abs(-10));
        System.out.println("Square Root: " + Math.sqrt(25));
        System.out.println("Power: " + Math.pow(2, 3));
        System.out.println("Max: " + Math.max(5, 10));
        System.out.println("Random: " + Math.random()); // Generates a random number
    }
}
```

Arrays in Java

An array is a collection of elements of the same data type, stored at contiguous memory locations. Arrays in Java are fixed in size and can store primitive data types as well as objects.

Declaring and Initializing Arrays

1D Array (One-Dimensional Array)

// Declaration

```
int[] arr = new int[5]; // Array of size 5
```

```
int[] arr2 = {10, 20, 30, 40, 50}; // Initialization with values
```

2D Array (Two-Dimensional Array)

// Declaration

```
int[][] matrix = new int[3][3];
```

// Initialization

```
int[][] matrix2 = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9} };

```


Accessing Elements in an Array

```
class ArrayExample
{
    public static void main(String args[])
    {
        int[] numbers = {10, 20, 30, 40, 50};

        // Access elements
        System.out.println("First Element: " + numbers[0]);
        System.out.println("Last Element: " + numbers[numbers.length - 1]);

        // Loop through the array
        System.out.print("Array Elements: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
    }
}
```

Output

First Element: 10

Last Element: 50

Array Elements: 10 20 30 40 50

Objects and Classes:

1. Class in Java

A class is a blueprint that defines attributes (variables) and behaviors (methods) of objects. It does not occupy memory until an object is created.

Example: Defining a Class

```
// Creating a class named Car
class Car
{
    // Attributes (fields)
    String brand;
    int speed;

    // Method to display car details
    void display()
    {
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed + " km/h");
    }
}
```

2. Object in Java

An object is an instance of a class. It has real values assigned to its attributes and can call methods defined in the class.

Example: Creating and Using an Object

```
public class Main {
    public static void main(String[] args) {
        // Creating an object of the Car class
        Car myCar = new Car();

        // Assigning values to the object's attributes
        myCar.brand = "Toyota";
        myCar.speed = 120;

        // Calling the display method
        myCar.display();
    }
}
```

Constructors in Java

A constructor is a special method in Java used to initialize objects. It has the same name as the class and does not have a return type. A constructor is automatically called when an object of the class is created.

Characteristics of Constructors in Java:

1. Same Name as the Class

A constructor must have the same name as the class in which it is defined. This helps the compiler recognize it as a constructor and not a regular method.

2. No Return Type

Constructors do not have a return type, not even void. Their purpose is to initialize objects rather than return a value.

3. Automatic Invocation

A constructor is automatically called when an object of the class is created, meaning there is no need to invoke it manually.

4. Used for Object Initialization

Constructors are primarily used to assign values to object attributes when the object is created. This ensures that objects are ready for use immediately.

5. Constructor Overloading

Multiple constructors can be defined in a class with different sets of parameters. This is known as constructor overloading, which allows flexibility in object creation.

6. Use of this Keyword

The this keyword can be used within a constructor to refer to the current object's attributes. It is helpful when parameter names conflict with instance variable names.

7. Constructor Chaining

One constructor can call another constructor within the same class using this(), reducing code duplication. This is called constructor chaining.

8. Cannot be Inherited

Constructors are not inherited by subclasses, but a subclass constructor can call a superclass constructor using super().

9. No Explicit Call Required

Since constructors are automatically invoked during object creation, there is no need to explicitly call them in the program.

10. Copy Constructor (Manual Implementation)

Java does not provide a built-in copy constructor, but it can be manually implemented to copy values from one object to another.

Types of Constructors in Java

Java provides three main types of constructors: Default Constructor, Parameterized Constructor, and Copy Constructor. Each serves a different purpose in object initialization.

1. Default Constructor

A default constructor is a constructor that does not take any parameters. It is automatically created by Java if no other constructor is defined. It initializes object attributes with default values (0 for numbers, null for objects, and false for booleans).

Example: Default Constructor

```
class Car {
    String brand;
    int speed;

    // Default Constructor
    Car() {
        brand = "Toyota";
        speed = 100;
    }

    void display() {
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // Calls the default constructor
        myCar.display();
    }
}
```

2. Parameterized Constructor

A parameterized constructor takes arguments to initialize an object with specific values. This allows customization of object attributes during object creation.

Example: Parameterized Constructor

```
class Car {
    String brand;
    int speed;
```

```

// Parameterized Constructor
Car(String b, int s) {
    brand = b;
    speed = s;
}

void display() {
    System.out.println("Brand: " + brand);
    System.out.println("Speed: " + speed + " km/h");
}
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Honda", 120); // Passing values to constructor
        myCar.display();
    }
}

```

3. Copy Constructor (Manual Implementation in Java)

A copy constructor creates a new object by copying the values of an existing object. Java does not provide a built-in copy constructor, but it can be implemented manually.

Example: Copy Constructor

```

class Car {
    String brand;
    int speed;

    // Parameterized Constructor
    Car(String b, int s) {
        brand = b;
        speed = s;
    }

    // Copy Constructor
    Car(Car obj) {
        brand = obj.brand;
        speed = obj.speed;
    }

    void display() {
        System.out.println("Brand: " + brand);
        System.out.println("Speed: " + speed + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car("BMW", 200); // Original object
        Car car2 = new Car(car1); // Copying values using copy constructor

        car1.display();
    }
}

```

```
        car2.display();
    }
}
```

Finalizer in Java

A finalizer in Java refers to the `finalize()` method, which is called by the garbage collector before an object is removed from memory. It is mainly used to perform cleanup operations like releasing system resources, closing files, or disconnecting databases before an object is destroyed.

However, the `finalize()` method is deprecated in Java 9 because it can cause performance issues and is not guaranteed to run immediately. Developers are encouraged to use `try-with-resources` or explicit cleanup methods instead.

The finalizer in Java is defined in the `Object` class, meaning every class inherits the `finalize()` method by default. It is primarily used for cleanup tasks, such as releasing system resources like files, network connections, or database connections. The garbage collector (JVM) calls `finalize()` before an object is destroyed, but its execution is not guaranteed to run immediately, as it depends on the garbage collection process, which operates periodically. Although `finalize()` can be overridden in a class to define custom cleanup logic, it is deprecated since Java 9 due to performance issues and unpredictable behavior. Instead of using `finalize()`, Java recommends `try-with-resources` for managing resources efficiently and explicit cleanup methods like `close()`.

```
class FinalizerDemo
{
    // Constructor
    FinalizerDemo()
    {
        System.out.println("Object Created");
    }

    // Overriding finalize() method
    @Override
    protected void finalize()
    {
        System.out.println("Finalize method called before object is garbage collected");
    }
}

public class Main
{
    public static void main(String[] args)
    {
        // Creating objects
        FinalizerDemo obj1 = new FinalizerDemo();
        FinalizerDemo obj2 = new FinalizerDemo();

        // Nullifying references, making objects eligible for garbage collection
        obj1 = null;
        obj2 = null;
    }
}
```

```

// Requesting garbage collection
System.gc();

System.out.println("End of program");
}
}

```

Visibility Modifiers in Java

In Java, visibility modifiers (also called access modifiers) define the scope of variables, methods, and classes. They help in encapsulation by restricting direct access to data, ensuring security, and controlling how different parts of a program interact. Java provides four visibility modifiers: private, default (no modifier), protected, and public. Each modifier determines whether a member can be accessed within the same class, package, subclass, or globally.

1. Private Modifier

The private modifier restricts access to within the same class only. It is the most restrictive access level, preventing access from outside the class, including subclasses and other classes in the same package. Private members are commonly used for data hiding, ensuring that class properties are not directly modified by external classes. However, they can be accessed indirectly through public getter and setter methods.

```

class Person {
    private String name = "John"; // Private variable

    private void display() { // Private method
        System.out.println("This is a private method.");
    }

    public void showName() { // Public method to access private data
        System.out.println("Name: " + name);
        display();
    }
}

public class Main {
    public static void main(String[] args) {
        Person obj = new Person();
        obj.showName();
    }
}

```

2. Default (No Modifier)

If no visibility modifier is specified, Java assigns default (package-private) access. This means members are accessible only within the same package but not from other packages or subclasses in different packages. This is useful for package-level encapsulation, where multiple classes within the same package need to communicate but should not be exposed globally.

```

class Example { // No modifier (default)
    int number = 42; // Default variable
}

```

```

        void display() { // Default method
            System.out.println("Default access: " + number);
        }
    }

    public class Main {
        public static void main(String[] args) {
            Example obj = new Example();
            obj.display();
        }
    }

```

3. Protected Modifier

The protected modifier allows access within the same package and also in subclasses (even in different packages). It is mostly used in inheritance, where a parent class provides access to its protected members for its child classes while still restricting access from non-subclass classes in other packages.

```

class Parent {
    protected String message = "Hello from Parent"; // Protected variable

    protected void greet() { // Protected method
        System.out.println("Greetings!");
    }
}

public class Main extends Parent {
    public static void main(String[] args) {
        Main obj = new Main();
        System.out.println(obj.message);
        obj.greet();
    }
}

```

4. Public Modifier

The public modifier provides unrestricted access to a class, method, or variable. It can be accessed from anywhere, including different packages and subclasses. This is useful for methods or variables that should be available globally, such as utility functions, API methods, or main program logic. However, using public should be done carefully to avoid unintentional modifications to data.

```

class PublicExample {
    public String message = "Public Access";

    public void display() { // Public method
        System.out.println("This is a public method.");
    }
}

public class Main {
    public static void main(String[] args) {
        PublicExample obj = new PublicExample();
    }
}

```

```

        System.out.println(obj.message);
        obj.display();
    }
}

```

Methods and objects

- In Java, **methods** are blocks of code that perform specific tasks, and **objects** are instances of classes that allow us to use these methods.
- Types of Methods:
 - Instance Methods - Work on objects.
 - Static Methods - Belong to the class and don't need an object.
 - Parameterized Methods - Accept inputs.
 - Return Type Methods - Return value

```

class Calculator
{
    // 1. Instance Method (Needs an object)
    void add(int a, int b)
    {
        System.out.println("Sum: " + (a + b));
    }
    // 2. Method with Return Type
    int multiply(int a, int b)
    {
        return a * b;
    }
    // 3. Static Method (No object needed)
    static void greet()
    {
        System.out.println("Hello! Welcome to Calculator");
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator obj = new Calculator(); // Creating an object

        obj.add(5, 3); // Calling instance method

        int result = obj.multiply(4, 2); // Storing return value
        System.out.println("Multiplication: " + result);
        Calculator.greet(); // Calling static method without an object
    }
}

```

Output

```

Sum: 8
Multiplication: 8
Hello! Welcome to Calculator

```

Inbuild class

- Java provides several built-in classes for handling text and characters efficiently. The most commonly used are:
- String Class - Immutable (cannot be changed after creation).
- Character Class - Used for working with single characters.
- StringBuffer Class - Mutable (can be modified after creation), thread-safe.

1. String Class (Immutable)

- String objects cannot be changed after they are created.
- When modified, a new object is created.

```
public class StringExample {
    public static void main(String[] args) {
        String str1 = "Hello"; // String literal
        String str2 = new String("World"); // Using new keyword
        System.out.println(str1 + " " + str2); // Concatenation
        System.out.println("Length: " + str1.length()); // Get length
        System.out.println("Uppercase: " + str1.toUpperCase()); // Convert to uppercase
    }
}
```

OUTPUT

Hello World

Length: 5 Uppercase: HELLO

2. Character Class (Wrapper for char)

- Used to work with single characters.
- Provides methods to check character properties.

```
public class CharacterExample {
    public static void main(String[] args) {
        char ch = 'A';
        System.out.println("Is Letter? " + Character.isLetter(ch)); // Checks if it's a letter
        System.out.println("Is Digit? " + Character.isDigit(ch)); // Checks if it's a digit
        System.out.println("Lowercase: " + Character.toLowerCase(ch)); // Converts to lowercase
    }
}
```

OUTPUT

Is Letter? True

Is Digit? false Lowercase: a

3. StringBuffer Class (Mutable)

- Unlike String, StringBuffer is mutable, meaning it can be changed without creating a new object.
- It is faster than String for modifications.

```
public class StringBufferExample {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");
```

```

        sb.append(" World"); // Append text
        System.out.println(sb);
        sb.insert(6, "Java "); // Insert text
        System.out.println(sb);
        sb.reverse(); // Reverse string
        System.out.println(sb);
    }
}

```

OUTPUT

```

Hello World
Hello Java World
dlroW avaJ olleH

```

Files

- Java provides the File class (in java.io package) to work with files. It allows us to create, delete, check existence, and get information about a file.

1. Creating a File

We use the `createNewFile()` method to create a file.

Example: Creating a File

```

import java.io.FileWriter;
import java.io.IOException;
public class FileWriteExample {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("test.txt");
            writer.write("Hello, Java!"); // Writing to file
            writer.close();
            System.out.println("File written successfully.");
        } catch (IOException e) {
            System.out.println("Error while writing to file.");
        }
    }
}

```

2. Reading a file

```

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
public class ReadFileScanner
{
    public static void main(String[] args)
    {
        try (Scanner sc = new Scanner(new File("test.txt")))
        {
            while (sc.hasNextLine())
            {
                System.out.println(sc.nextLine());
            }
        }
        catch (FileNotFoundException e)

```

```

    {
        System.out.println(" File not found!");
    }
}
}

```

3.Deleting a File

- We use delete() to remove a file.

```

import java.io.File;
public class FileDeleteExample {
    public static void main(String[] args) {
        try {
            File file = new File("test.txt");
            if (file.delete()) {
                System.out.println(" File deleted successfully.");
            } else {
                System.out.println("File not found or cannot be deleted.");
            }
        } catch (Exception e) {
            System.out.println("Error while deleting file.");
        }
    }
}

```

this Reference

- Why Use this?
- To differentiate instance variables from local variables (when they have the same name).
- To call another constructor inside the same class.
- To call a method inside the same class.

```

class Student {
    String name;
    Student(String name) {
        this.name = name; // `this` refers to the instance variable
    }
    void display() {
        System.out.println("Student Name: " + this.name);
    }
}

public class ThisExample {
    public static void main(String[] args) {
        Student s1 = new Student("John");
        s1.display(); // Output: Student Name: John
    }
}

```

Java I/O Streams

Java I/O Streams are used to handle input and output operations in Java. They allow

reading from and writing to different data sources such as files, keyboard input, or network sockets.

Types of Streams in Java

1. Read Stream (InputStream/Reader)

Read streams are used to read data from a source (file, console, etc.).

Example: Reading from a File Using FileInputStream

```
import java.io.FileInputStream;
```

```
import java.io.IOException;
```

```
public class ReadExample {  
    public static void main(String[] args) {  
        try {  
            FileInputStream file = new FileInputStream("input.txt");  
            int i;  
            while ((i = file.read()) != -1) {  
                System.out.print((char) i); // Reading file content character by character  
            }  
            file.close();  
        } catch (IOException e) {  
            System.out.println("Error reading file: " + e.getMessage());  
        }  
    }  
}
```

2. Write Stream (OutputStream/Writer)

Write streams are used to write data to a destination.

Example: Writing to a File Using FileOutputStream

```
import java.io.FileOutputStream;
```

```
import java.io.IOException;
```

```
public class WriteExample {  
    public static void main(String[] args) {  
        String text = "Hello, Java I/O!";  
        try {  
            FileOutputStream file = new FileOutputStream("output.txt");  
            file.write(text.getBytes()); // Writing string to file  
            file.close();  
            System.out.println("Data written successfully!");  
        } catch (IOException e) {  
            System.out.println("Error writing file: " + e.getMessage());  
        }  
    }  
}
```

3. Buffered Stream (BufferedInputStream/BufferedReader)

Buffered streams improve performance by reducing the number of I/O operations.

Example: Reading from a File Using BufferedReader

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class BufferedReadExample {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(new FileReader("input.txt"));
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line); // Reading file line by line
            }
            br.close();
        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

SAHYADRI INSTITUTE OF COMMERCE